

Assignment –1

Leetcode Exercise

1. Big Countries

Problem Summary

You are given a table World with the following columns:

- name (country name)
- continent
- area (in square km)
- population
- gdp

A country is considered “big” if:

- Its area $\geq 3,000,000$, OR
- Its population $\geq 25,000,000$

Task

Write an SQL query to return the name, population, and area of all big countries.

SQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT name, population, area
3 FROM World
4 WHERE area >= 3000000 OR population >= 25000000;
```

Output

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Expected

name	population	area
Afghanistan	25500100	652230
Algeria	37100000	2381741

Explanation

- The SELECT clause retrieves required columns.
- The WHERE clause filters records based on the given conditions.
- The OR operator ensures that even if one condition is satisfied, the country is Included.

2.. Find Customer Referee

Problem Summary

You are given a table Customer with the following columns:

- id (customer ID)
- name (customer name)
- referee_id (ID of the referring customer)

Task

Find the names of customers who were NOT referred by the customer with id = 2.

- Note: Customers with NULL in referee_id should also be included.

SQL Solution

```
MySQL v Auto

1 # Write your MySQL query statement below
2 SELECT name
3 FROM Customer
4 WHERE referee_id != 2 OR referee_id IS NULL;
```

Output

```
| name |
| ---- |
| Will |
| Jane |
| Bill |
| Zack |
```

Expected

```
| name |
| ---- |
| Will |
| Jane |
| Bill |
| Zack |
```

Explanation

- referee_id != 2 → selects customers not referred by ID 2
- referee_id IS NULL → includes customers with no referee
- OR ensures both conditions are considered

3. Not Boring Movies

Problem Summary

You are given a table Cinema with the following columns:

- id (movie ID)
- movie (movie name)
- description (movie type)
- rating (movie rating)

Task

Find all movies that are:

- Not boring → description != 'boring'
- Have odd-numbered IDs → id % 2 = 1

Return the result sorted by rating in descending order.

SQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT *
3 FROM Cinema
4 WHERE description != 'boring'
5 AND id % 2 = 1
6 ORDER BY rating DESC;
```

Output

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

Expected

id	movie	description	rating
5	House card	Interesting	9.1
1	War	great 3D	8.9

Explanation

- description != 'boring' → filters out boring movies
- id % 2 = 1 → selects only odd-numbered IDs
- ORDER BY rating DESC → sorts results from highest to lowest rating

4. Invalid Tweets

Problem Summary

You are given a table Tweets with the following column:

- tweet_id (ID of the tweet)

- content (text of the tweet)

Task

Find the IDs of tweets that are invalid, where a tweet is considered invalid if:

- The length of the content is greater than 15 characters

SQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT tweet_id
3 FROM Tweets
4 WHERE LENGTH(content) > 15;
```

Output

tweet_id
2

Expected

tweet_id
2

Explanation

- LENGTH(content) calculates the number of characters in the tweet
- > 15 filters tweets that exceed the allowed length

- Only tweet_id is returned as required

5. Recyclable and Low-Fat Products

Problem Summary

You are given a table Products with the following columns:

- product_id
- low_fats (values: 'Y' or 'N')
- recyclable (values: 'Y' or 'N')

Task

Find the product IDs of products that are:

- Low fat (low_fats = 'Y')
- Recyclable (recyclable = 'Y')

SQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT product_id
3 FROM Products
4 WHERE low_fats = 'Y'
5 AND recyclable = 'Y';
```

Output

```
| product_id |
|-----|
| 1          |
| 3          |
```

Expected

```
| product_id |
|-----|
| 1          |
| 3          |
```

Explanation

- `low_fats = 'Y'` → selects only low-fat products
- `recyclable = 'Y'` → selects recyclable products
- `AND` ensures both conditions are satisfied

6. Article Views I

Problem Summary

You are given a table `Views` with the following columns:

- `article_id`
- `author_id`
- `viewer_id`
- `view_date`

Task

Find all authors who viewed at least one of their own articles.

- Return the result as a list of unique author IDs
- Sort the result in ascending order

SQL Solution

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT DISTINCT author_id AS id
3 FROM Views
4 WHERE author_id = viewer_id
5 ORDER BY id ASC;
```

Output

id
4
7

Expected

id
4
7

Explanation

- `author_id = viewer_id` → identifies cases where authors viewed their own articles
- `DISTINCT` → removes duplicate author IDs
- `ORDER BY id` → sorts the result in ascending order

